

Parker Lovill
ID#: 00127715
08/03/2026
CS514

Assignment - Introduction to the Untyped Lambda Calculus

Use the following syntax:

$$t ::= x \mid \lambda x. t \mid t t$$

For OCaml questions, use:

type term =

| Var of string

| Abs of string * term

| App of term * term

Question 1 — Term Forms

For each term, identify its outermost form as a Variable, Abstraction, or Application.

1. x **Variable**
2. $\lambda x. x$ **Abstraction**
3. $(\lambda x. x) y$ **Application**
4. $\lambda x. (x y)$ **Abstraction**

Question 2 - Free Variables

List the free variables in:

$$\lambda x. (x y)$$

{y}

Question 3 - More Free Variables

List the free variables in:

$$\lambda x. \lambda y. (x\ z)$$

{z}

Question 4 - Scope and Shadowing

Consider:

$$\lambda x. \lambda x. x$$

Which lambda abstraction binds the final x? Explain briefly.

The second λx binds the last x

There are two x's here with lambdas, so the closest one is picked. The inner λx is closer to that final x, so it takes it first.

Question 5 - Alpha-Equivalence

Are the following terms alpha-equivalent? Explain briefly.

$$\lambda x. \lambda y. x$$

and:

$$\lambda a. \lambda b. a$$

Yes they are alpha-equivalent

The pattern here is what you look at,

$\lambda x. \lambda y. x$ and $\lambda a. \lambda b. a$ do the exact same thing. They take two inputs and give back the first one.

Question 6 - Not Alpha-Equivalent

Are the following terms alpha-equivalent? Explain why or why not.

$$\lambda x. \lambda y. x$$

and:

$$\lambda x. \lambda y. y$$

These are not alpha-equivalent

$\lambda x. \lambda y. x$ The body is bound by the outer λx

$\lambda x. \lambda y. y$ The body is bound by the inner λy

Question 7 - OCaml AST

Translate:

$$\lambda x. (x\ y)$$

into OCaml using the term type above.

Abs ("x", App (Var "x", Var "y"))

Question 8 - OCaml AST with Nested Functions

Translate:

$$\lambda x. \lambda y. x$$

into OCaml.

Abs ("x", Abs ("y", Var "x"))

Question 9 - Simple Beta-Reduction

Reduce:

$$(\lambda x.x)(\lambda y.y)$$

Show the substitution step.

$$(\lambda x.x)(\lambda y.y) =$$

$$[x \rightarrow \lambda y.y]x = y.y$$

Question 10 - Beta-Reduction with a Nested Function

Reduce:

$$(\lambda x.\lambda y.x)(\lambda z.z)$$

$$(\lambda x.\lambda y.x)(\lambda z.z) =$$

$$[x \rightarrow \lambda z.z](\lambda y.x) =$$

$$\lambda y.(\lambda z.z)$$

Question 11 - Multi-Step Reduction

Reduce completely:

$$(\lambda x.x)((\lambda y.y)(\lambda z.z))$$

Show each step.

$$(\lambda x.x)((\lambda y.y)(\lambda z.z))$$

$$(\lambda x.x)(\lambda z.z) \quad (\lambda y.y)(\lambda z.z) = \lambda z.z$$

$$\lambda z.z \quad (\lambda x.x) \text{ returns back}$$

Question 12 - Basic Substitution

Compute:

$$[x \mapsto y](x z)$$

Answer: $y z$

Question 13 - Bound Variables and Substitution

Compute:

$$[x \mapsto y](\lambda x. x)$$

Answer: $\lambda x. x$

Explain why the term does or does not change.

Question 14 - Variable Capture

Consider:

$$[x \mapsto y](\lambda y. x)$$

1. What incorrect result would naive text replacement produce?

$\lambda y. y$

2. Why is that result incorrect?

The y being plugged in is supposed to be a free variable outside.

The λy takes it and binds to it.

3. Alpha-rename the bound y to z and perform the substitution correctly.

Rename: $\lambda y. x \rightarrow \alpha \lambda z. x$

Substitution: $\lambda z. y$

Question 15 - Small OCaml Programming Exercise

Write both of the following functions.

A. Count AST Node

`sval` size : term -> int

Count each Var, Abs, and App node as one node.

For example:

size (Abs ("x", Var "x"))

should return:

2

```
let rec size (t : term) : int =  
  match t with  
  | Var _ -> 1  
  | Abs (_, t1) -> 1 + size t1  
  | App (t1, t2) -> 1 + size t1 + size t2
```

B. Determine Whether a Term Is a Value

For this assignment, only abstractions are values.

Write:

```
val is_value : term -> bool
```

For example:

```
is_value (Abs ("x", Var "x"))
```

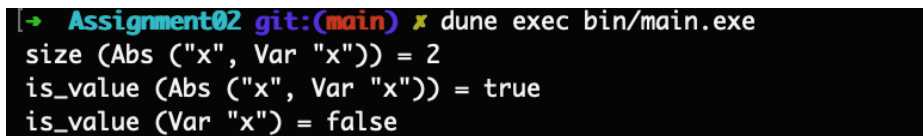
should return true, while:

```
is_value (Var "x")
```

should return false.

```
let is_value (t : term) : bool =  
  match t with  
  | Abs (_, _) -> true  
  | _ -> false
```

Output screenshot:



```
➔ Assignment02 git:(main) ✗ dune exec bin/main.exe  
size (Abs ("x", Var "x")) = 2  
is_value (Abs ("x", Var "x")) = true  
is_value (Var "x") = false
```